So far it’s been all individual modules, but sometimes you’ll want to include a whole group of modules. In that case you’ll need a module index.

12.1.6. Indexes

When you have many modules, it’s easier to load several of them together instead of loading them all individually. Node.js supports module indexes to help you with this.[2] An index file is the module you get back when you require a directory.

[2] You can also create Node.js packages with npm, but again, that’s beyond the scope of this book.

Suppose you have a set of modules that you keep under a directory named utils:

```
├── utils/
│   ├── string.coffee
│   ├── array.coffee
│   ├── statistics.coffee
```

If you want to use several of those utility modules from another module, having to do multiple `require` statements is tedious:

```
{trim, pad} = require 'utils/string'
{remove} = require 'utils/array'
{chebyshev} = require 'utils/stats'
```

To avoid that tedium, create an intermediary index file from which you can load all of the other modules. An index file goes in the same directory as the other modules and does both `require` and `exports` for each of them:

```
exports.string = require '.utils/string'
exports.array = require '.utils/array'
exports.stats = require '.utils/stats'
```

Now when you require the `utils/index` module, you can destructure the object returned by `require` in a single line: